

Machine Learning Portfolio: Classification, Clustering, Reinforcement Learning, and Deep Learning

Name: Abdulaziz Hameed Aloufi

Student ID: C00266252

Module: Data Science & Machine Learning 2

Email: mr.alofi19@gmail.com

GitHub: <https://github.com/Abdulaziz1313/machine-learning-portfolio>

Date: 17/04/2026

Contents

1. Introduction
2. Dataset and Environment Description
3. Data Preparation and Preprocessing
4. K-Nearest Neighbour
5. K-Means Clustering
6. Support Vector Machine
7. Neural Networks
8. Reinforcement Learning
9. Deep Learning
10. Discussion
11. Conclusion
12. References

1. Introduction

Machine learning provides a variety of techniques for analysing data, recognising patterns, making predictions, grouping observations, and learning through interaction with an environment. This portfolio presents a range of practical machine learning implementations developed in Python using Jupyter notebooks.

Rather than focusing on only one model, this portfolio includes supervised learning, unsupervised learning, neural networks, reinforcement learning, and deep learning. The aim is to demonstrate understanding of several core machine learning topics through practical implementation, evaluation, and reflection.

Most of the notebooks in this portfolio use a dataset about the impact of AI on student performance. This dataset makes it possible to explore classification and clustering problems using academic and AI-related features. In addition, the reinforcement learning section uses the FrozenLake environment from Gymnasium, since reinforcement learning requires an interactive environment rather than a standard labelled dataset.

The portfolio includes the following notebook implementations:

- k-Nearest Neighbour (KNN)
- k-Means clustering
- Support Vector Machine (SVM)
- Artificial Neural Networks
- Reinforcement Learning with Q-learning
- Deep Learning with TensorFlow/Keras

Together, these notebooks show different styles of machine learning and highlight how model choice, preprocessing, parameter tuning, and evaluation affect results.

2. Dataset and Environment Description

2.1 AI and Student Performance Dataset

The main dataset used in this portfolio is a dataset about the impact of AI on student performance. It contains 8,000 rows and 26 columns. The target used in the classification notebooks is `performance_category`, with three classes: **Low**, **Medium**, and **High**. The dataset includes academic, behavioural, and AI-related features such as study hours, attendance, class participation, AI usage, and AI prompts per week.

The dataset contains missing values in the categorical variables `ai_tools_used` and `ai_usage_purpose`. These were handled by replacing missing values with the label "Unknown".

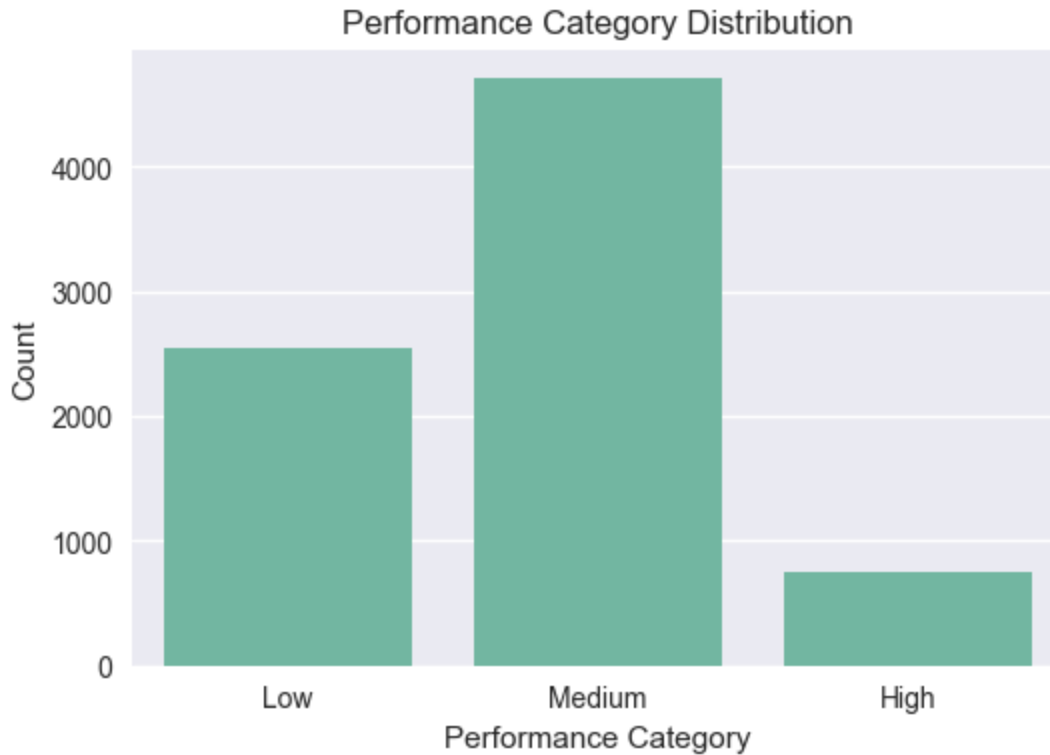


Figure 1. Performance category distribution in the AI and student performance dataset.

Figure 1 shows the distribution of the target variable. The Medium class is the largest group in the dataset, followed by Low, while High has the fewest examples. This class imbalance helps explain why the High class is more difficult to classify correctly in several of the supervised learning notebooks.

2.2 FrozenLake Environment

For the reinforcement learning notebook, the portfolio uses the FrozenLake environment from Gymnasium. In this environment, an agent must move from the start state to the goal without falling into holes. The environment is useful for demonstrating states, actions, rewards, exploration, exploitation, and Q-learning in a simple and interpretable way.

Unlike the other notebooks, this section does not use a static labelled dataset. Instead, the model learns through interaction with the environment over many episodes.

3. Data Preparation and Preprocessing

A consistent preprocessing workflow was applied across the classification, clustering, neural network, and deep learning notebooks. The main steps included:

- loading the dataset into pandas
- checking dataset shape and column names
- identifying missing values
- filling missing values in `ai_tools_used` and `ai_usage_purpose` with "Unknown"
- selecting suitable input features
- removing columns that should not be used for prediction, such as:
 - `student_id`
 - `final_score`
 - `passed`
 - `performance_category` where appropriate
- encoding categorical variables using `LabelEncoder`
- splitting data into training and test sets for supervised models
- scaling features using `StandardScaler` where needed

Scaling was especially important for KNN, SVM, neural networks, and deep learning, because these models are sensitive to feature magnitude. Encoding categorical variables was also necessary because machine learning algorithms cannot directly process string-based categories.

These steps ensured that the data was clean, numeric, and suitable for the machine learning methods used in the portfolio.

4. K-Nearest Neighbour

The first supervised learning notebook implements the k-Nearest Neighbour algorithm. KNN is a distance-based classifier that predicts the class of a new sample based on the labels of its nearest neighbours.

In this notebook, the aim was to classify student performance category as Low, Medium, or High using AI usage and academic-related features. After preprocessing, encoding, splitting, and scaling, an initial KNN model was trained using $k = 5$. The first model achieved an accuracy of **0.7031**.

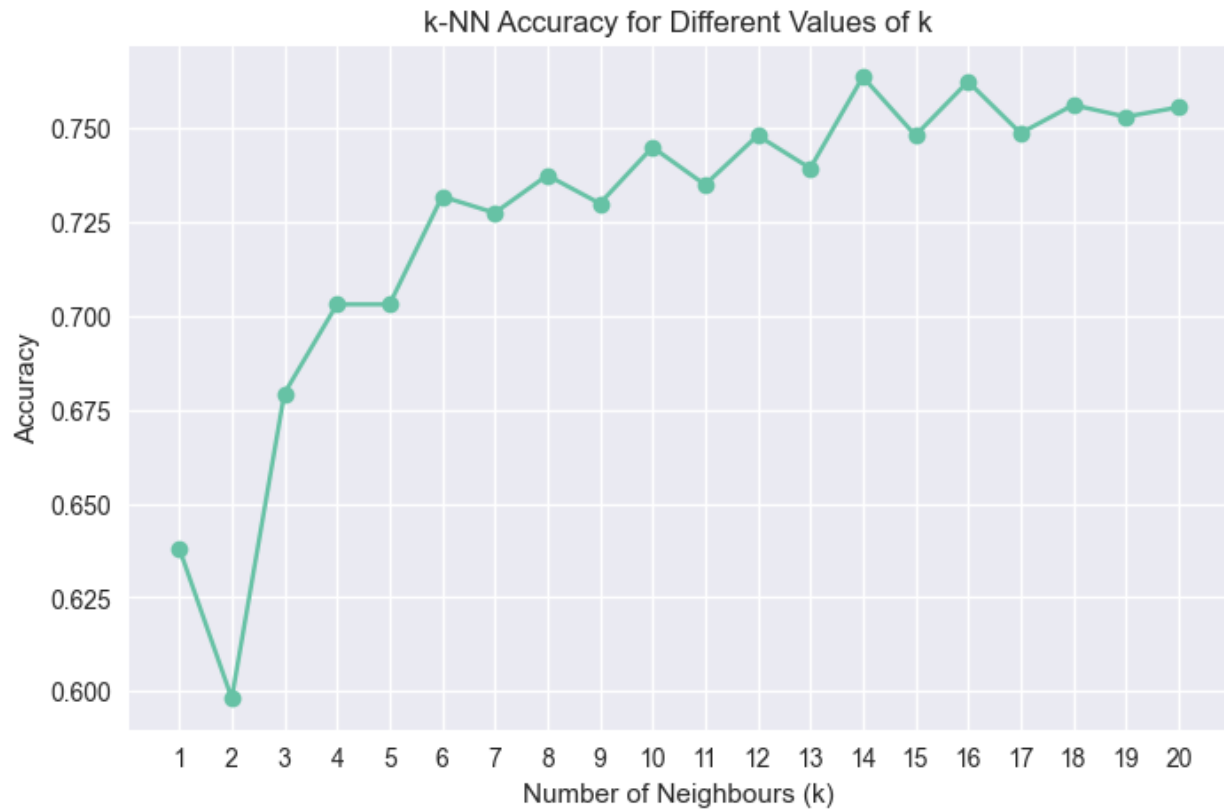


Figure 2. KNN accuracy for different values of k.

To improve performance, multiple values of k were tested from 1 to 20. As shown in Figure 2, model accuracy improved as larger values of k were tested, with the best result appearing at $k = 14$.

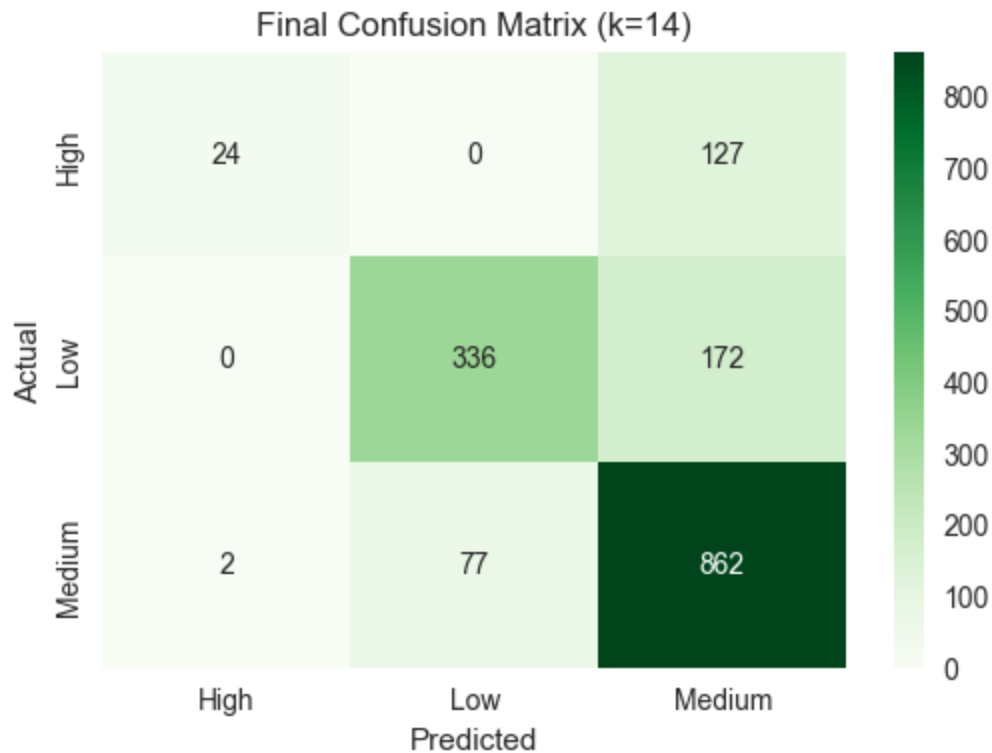


Figure 3. Final confusion matrix for the KNN model with $k = 14$.

The final KNN model with $k = 14$ achieved an accuracy of **0.7638**. Figure 3 shows the confusion matrix for this final model. The model performed best on the Medium class, followed by Low, while the High class remained the hardest to classify correctly. This is likely influenced by the class imbalance in the dataset.

This notebook demonstrated two important ideas: first, KNN depends strongly on the choice of k , and second, feature scaling is essential for distance-based models.

5. K-Means Clustering

The second notebook implements k-Means clustering as an unsupervised learning method. Unlike classification models, k-Means does not use labels during training. Instead, it attempts to group observations into clusters based on similarity.

For clustering, the columns `student_id`, `performance_category`, `passed`, and `final_score` were removed before training, since they either identified the rows directly or contained output information that should not influence cluster formation.

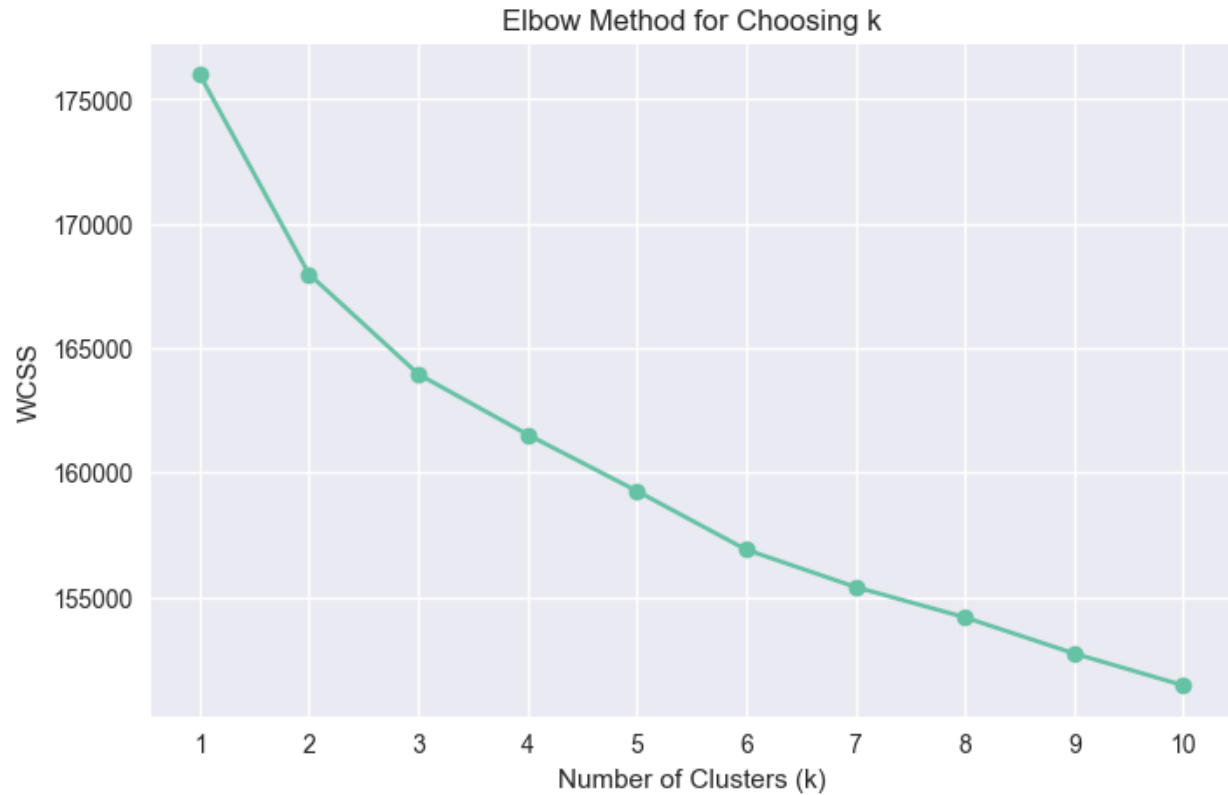


Figure 4. Elbow method used to choose the number of clusters for k-Means.

After encoding categorical variables and scaling the features, the elbow method was used to inspect possible values of k . Figure 4 shows the elbow curve. Based on this plot, $k = 3$ was chosen as a reasonable clustering solution.

	performance_category	cluster
0	Low	0
1	Medium	0
2	Medium	2
3	Medium	2
4	Low	1

Figure 5. Cross-tabulation between discovered clusters and known performance categories.

The final cluster sizes were:

- Cluster 0: 2509
- Cluster 1: 2619
- Cluster 2: 2872

Figure 5 compares the discovered clusters with the known `performance_category` labels. The results show that the clusters did not strongly separate the Low, Medium, and High groups. All three clusters were still dominated by the Medium class.

However, the cluster summary suggested that AI usage behaviour may be one of the strongest natural dividing lines in the data. This means that k-Means still revealed meaningful structure, even if it did not align closely with the formal performance labels.

This notebook demonstrated that unsupervised learning can reveal patterns that differ from the explicit targets used in supervised learning.

6. Support Vector Machine

The third notebook implements Support Vector Machine classification. SVM is a strong supervised learning method that can perform well on tabular data, especially when the features are scaled appropriately.

After preprocessing, encoding, scaling, and train-test splitting, an initial SVM model using the RBF kernel achieved an accuracy of **0.8413**.

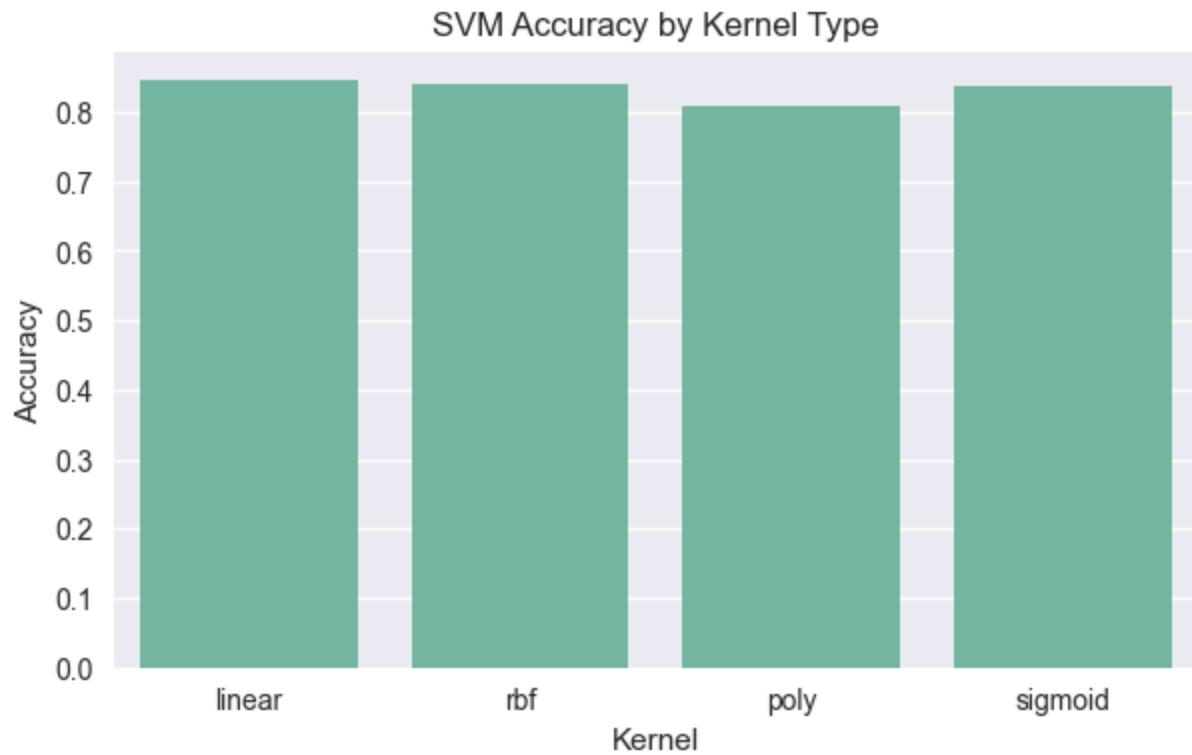


Figure 6. SVM accuracy by kernel type.

To investigate whether performance could be improved, four kernels were compared:

- linear
- rbf
- poly
- sigmoid

As shown in Figure 6, the linear kernel produced the best result.

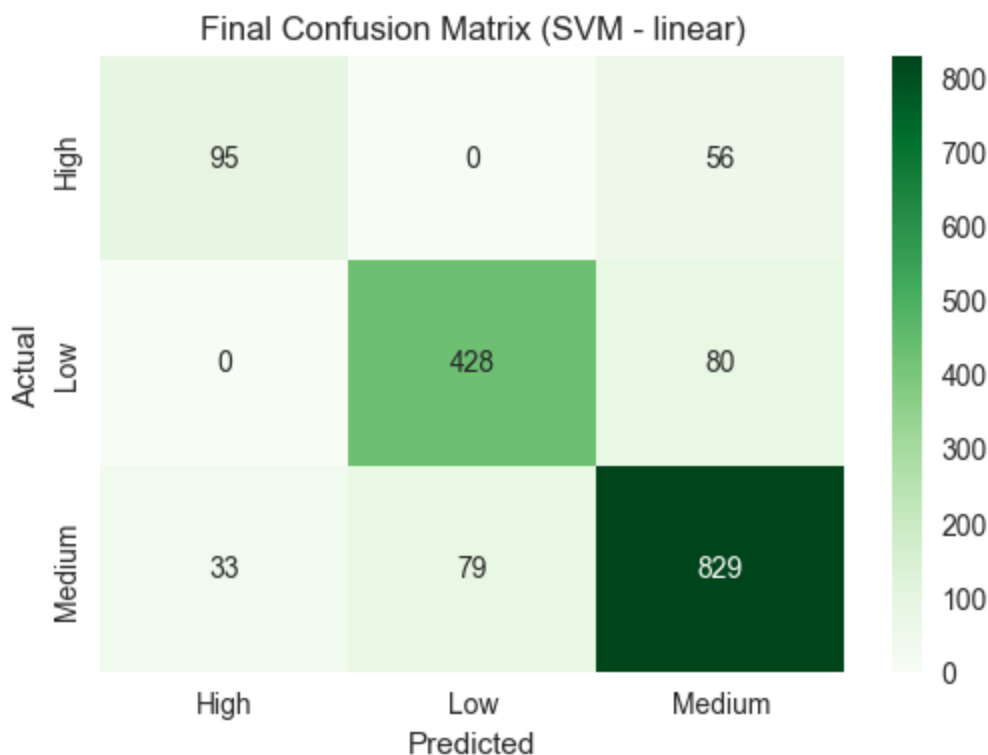


Figure 7. Final confusion matrix for the SVM model with the linear kernel.

The final linear SVM model achieved an accuracy of **0.8450**. Figure 7 shows the final confusion matrix. Among the classical supervised methods in the portfolio, SVM produced the strongest overall result.

As with the KNN notebook, performance was strongest on the Medium and Low classes, while the High class remained harder to classify accurately. This notebook showed that kernel selection affects performance and that a simpler linear kernel can sometimes outperform more complex alternatives.

7. Neural Networks

The fourth notebook implements an Artificial Neural Network using `MLPClassifier`. This notebook extends the portfolio beyond classical machine learning and into neural-network-based classification.

After preprocessing, encoding, scaling, and train-test splitting, an initial ANN with one hidden layer of 50 units achieved an accuracy of **0.8169**.

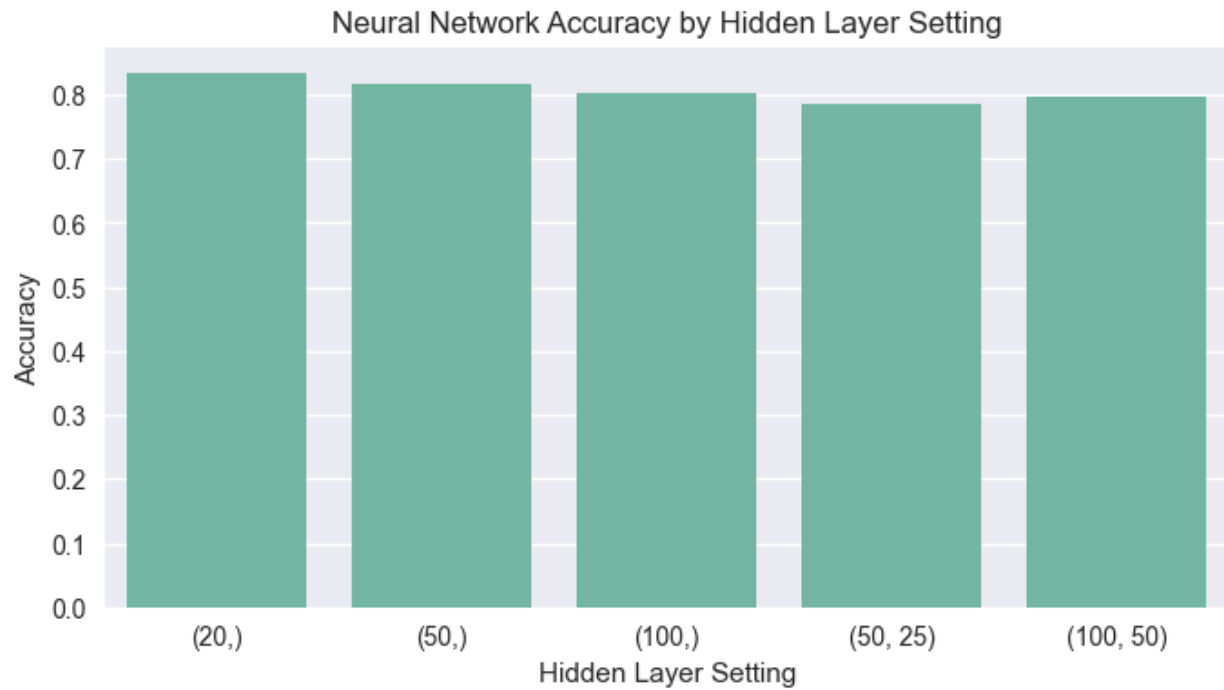


Figure 8. Accuracy of the ANN model for different hidden layer settings.

Several hidden layer settings were then compared. Figure 8 summarises the architecture comparison. The best-performing network structure was **(20,)**, which achieved a final accuracy of **0.8331**.

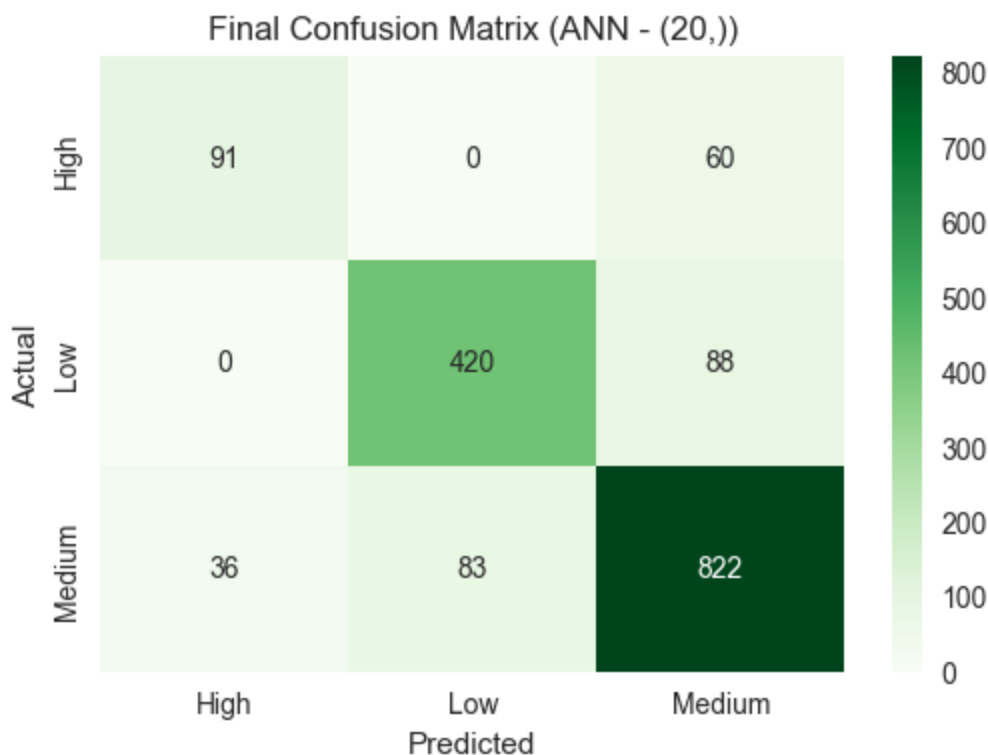


Figure 9. Final confusion matrix for the ANN model with architecture (20,).

Figure 9 shows the final confusion matrix for the selected ANN. The model performed well on the Medium and Low classes, with weaker performance on the High class.

This notebook showed that larger neural network architectures do not automatically improve results. In this case, the simplest tested model produced the strongest performance, which highlights the importance of comparing architectures rather than assuming that more complexity is always better.

8. Reinforcement Learning

The fifth notebook implements reinforcement learning using Q-learning on the FrozenLake environment.

The notebook begins by creating the environment, identifying the number of states and actions, and initialising a Q-table. Learning parameters such as learning rate, discount factor, epsilon, epsilon decay, and number of episodes were then defined.

The first training setup failed completely. The Q-table remained at zero, the success rate stayed at zero, and the agent did not learn the correct path. This was an important development step because it showed that reinforcement learning is highly dependent on parameter tuning.

To improve the model, several changes were made:

- `epsilon_decay` was slowed from 0.995 to 0.999
- `num_episodes` was increased from 2000 to 10000
- `epsilon_min` was increased from 0.01 to 0.05
- a small random tie-breaking factor was added to the action selection step

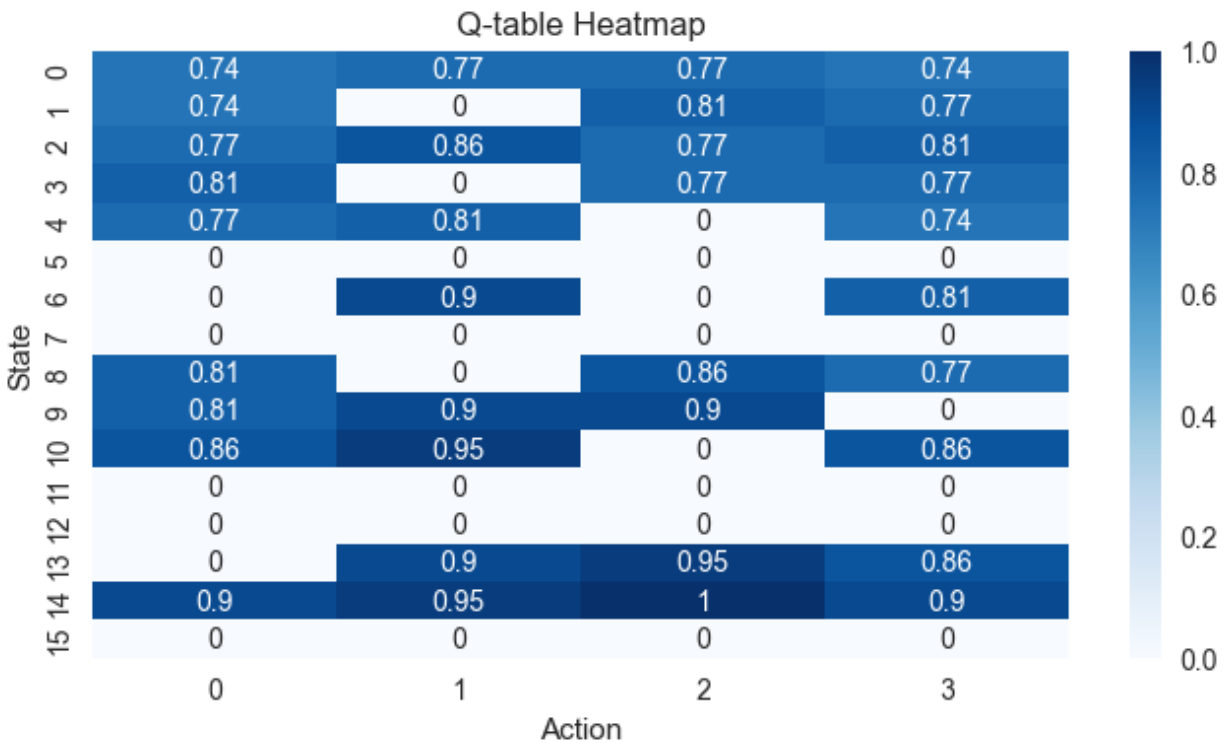


Figure 10. Learned Q-table after Q-learning training on FrozenLake.

Figure 10 shows the learned Q-table after training. The values are no longer zero, which indicates that the agent learned useful action preferences in different states.

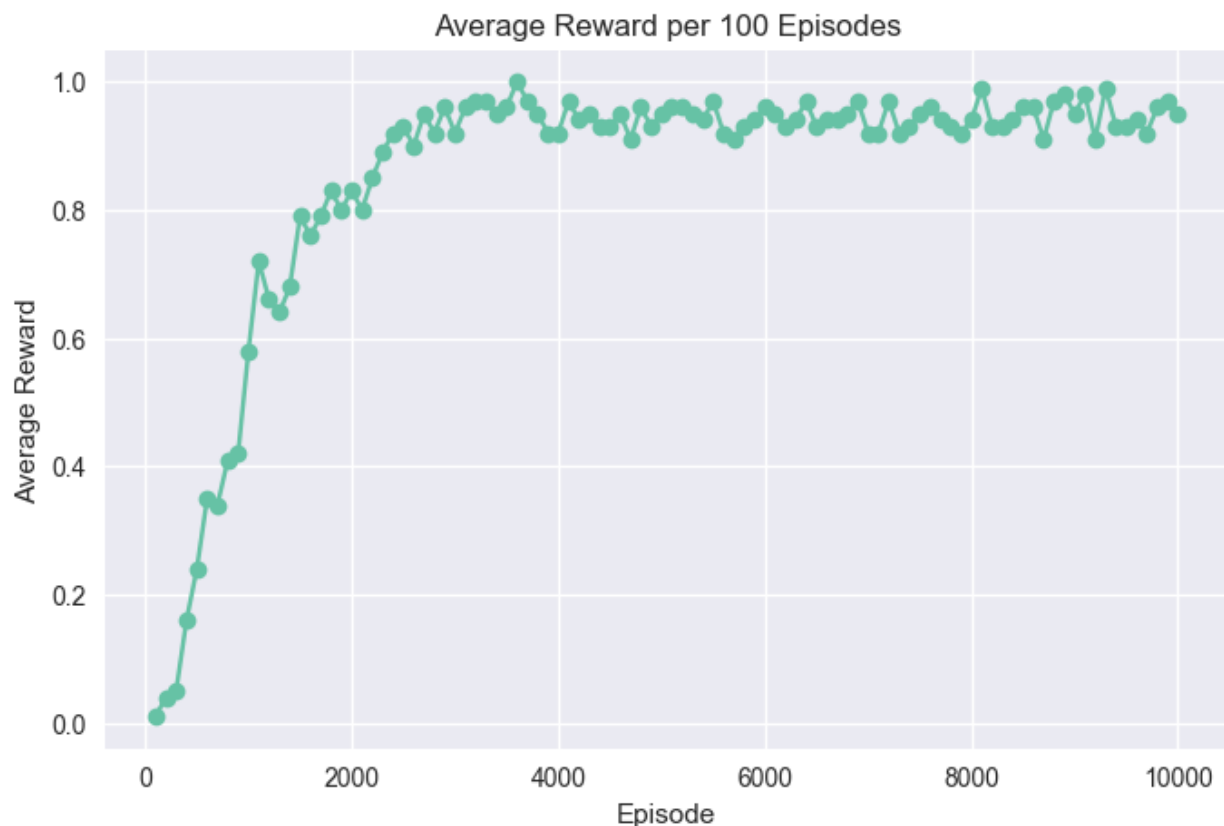


Figure 11. Average reward per 100 episodes during reinforcement learning training.

Figure 11 shows the reward curve over time. The graph rises steadily and then remains high, indicating that the agent improved significantly during training.

The final results were:

- successful training episodes: **8572**
- overall success rate: **0.8572**
- average test reward: **1.0**

In the final testing stage, the agent succeeded in all 20 test episodes.

This notebook differs from the rest of the portfolio because the model learns through interaction, rewards, and repeated trial-and-error rather than from labelled examples.

9. Deep Learning

The sixth notebook implements a deep learning model using TensorFlow/Keras. This notebook extends the portfolio from classical machine learning and basic neural networks into a deeper learning framework.

After preprocessing, encoding, scaling, and one-hot encoding of the target labels, an initial dense neural network was trained. This first model achieved a test accuracy of **0.8425**.

Its class-level performance was:

- High: precision **0.74**, recall **0.67**, f1-score **0.70**
- Low: precision **0.83**, recall **0.84**, f1-score **0.84**
- Medium: precision **0.86**, recall **0.87**, f1-score **0.87**

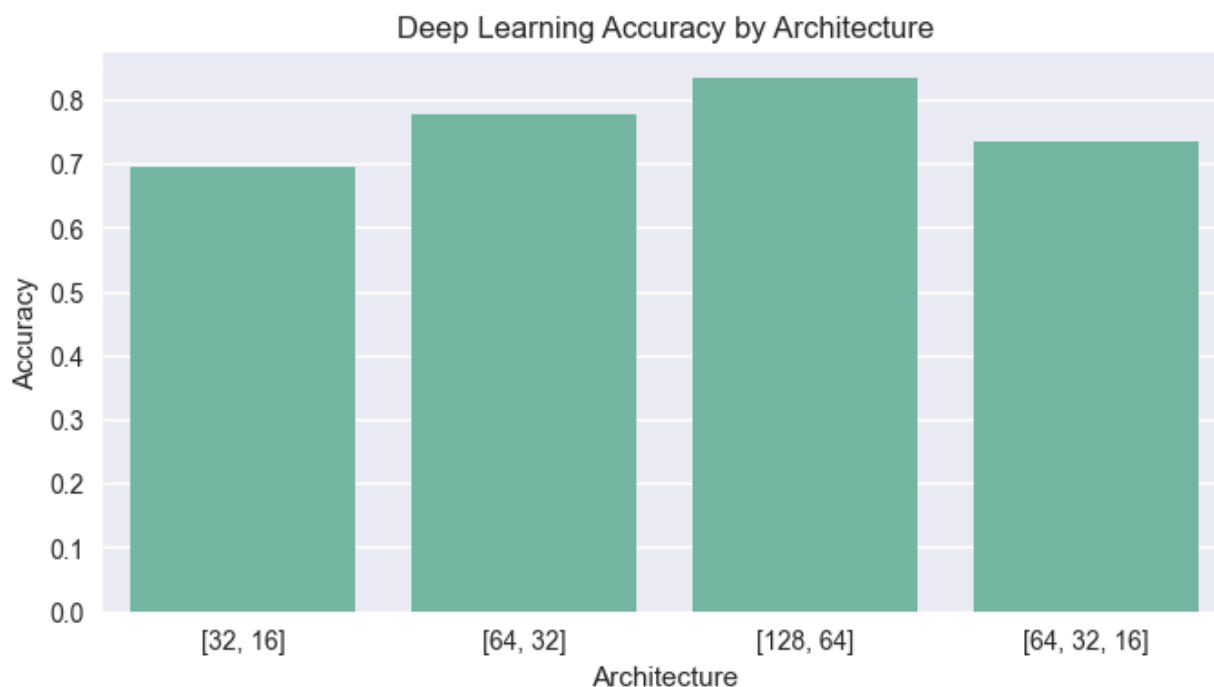


Figure 12. Deep learning accuracy for different architectures.

Several alternative architectures were then compared. Figure 12 shows the architecture comparison. The best alternative structure was **[128, 64]**, with an accuracy of **0.8244**, but this still did not outperform the initial model.

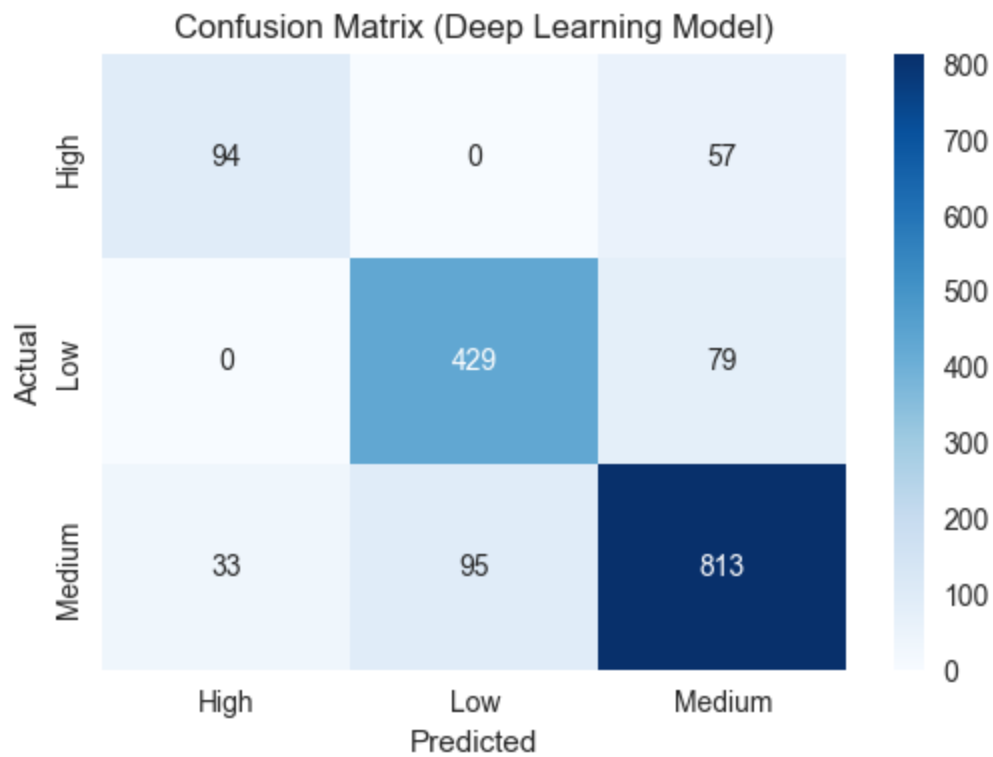


Figure 13. Confusion matrix for the initial deep learning model.

Figure 13 shows the confusion matrix for the initial deep learning model, which was the strongest result in this notebook.

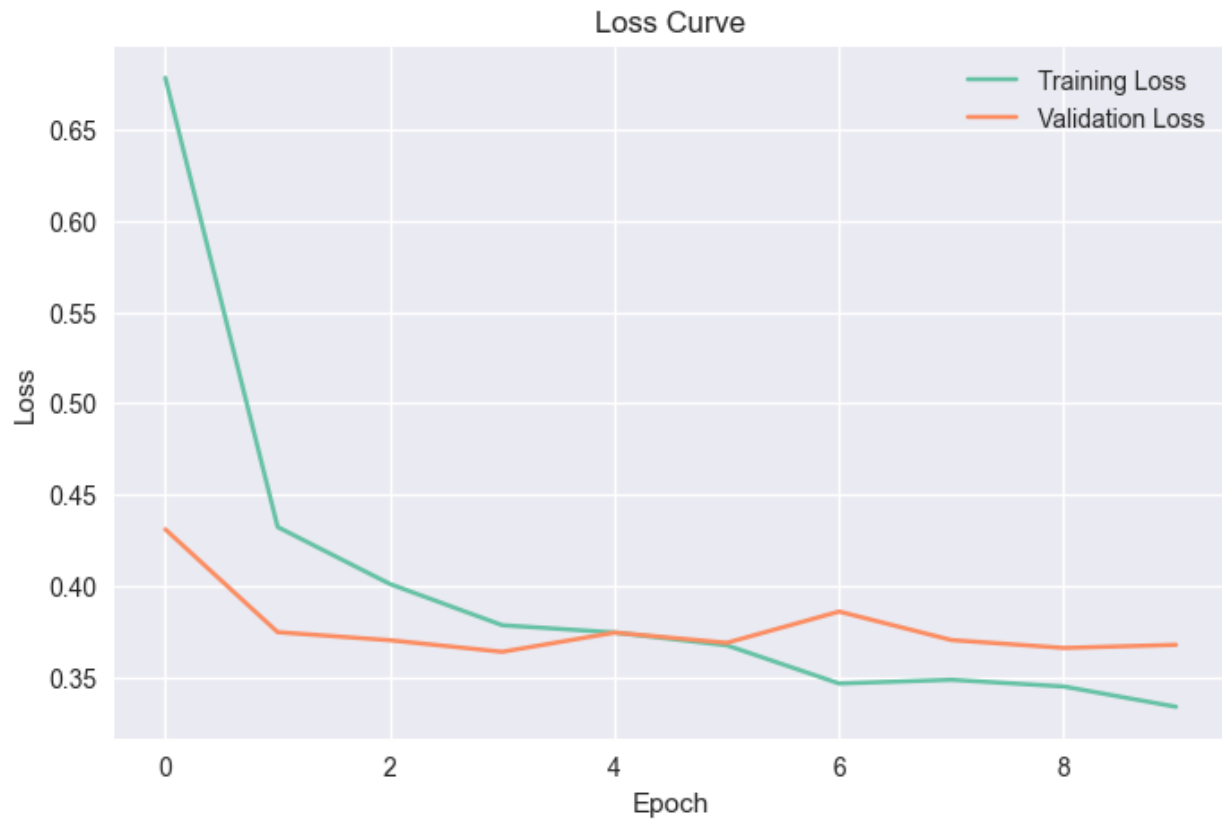


Figure 14. Training and validation loss curve for the deep learning model.

Figure 14 shows the loss curve. Both training and validation loss decrease early in training and then stabilise, which suggests that the model learned useful patterns without severe instability.



Figure 15. Training and validation accuracy curve for the deep learning model.

Figure 15 shows the accuracy curve. Training and validation accuracy both rise over time, with validation performance remaining relatively stable. This suggests that the model generalised reasonably well.

This notebook shows that deep learning can perform strongly on the dataset, but also that larger architectures do not necessarily improve results. It also confirms that this dataset is more suitable for a dense feedforward model than for a transformer-based approach, since the data is tabular rather than text- or sequence-based.

10. Discussion

This portfolio demonstrates several important ideas about machine learning in practice.

First, different algorithms behave differently even when they are applied to the same underlying data. KNN provided a useful baseline, SVM produced the strongest classical supervised result, neural networks performed competitively, and deep learning gave strong but slightly variable results.

Second, preprocessing had a major effect on performance. Missing value handling, encoding, feature selection, and scaling were all necessary for the models to work properly.

Third, larger or more complex models do not automatically guarantee better performance. This was seen in both the neural network and deep learning notebooks, where smaller or earlier models sometimes outperformed more complex alternatives.

Fourth, unsupervised learning and reinforcement learning revealed different aspects of machine learning. K-Means showed that the natural structure in the data did not strongly align with the labelled performance categories, while reinforcement learning demonstrated how an agent can learn through exploration and reward rather than from labelled examples.

A limitation of the portfolio is that most of the notebooks rely on the same main dataset. However, this also makes comparison across models clearer and more controlled. Future work could include additional datasets, deeper hyperparameter tuning, and more advanced evaluation approaches.

11. Conclusion

This portfolio presents a broad set of machine learning implementations covering classification, clustering, neural networks, reinforcement learning, and deep learning.

The work demonstrates practical skills in:

- data preparation
- preprocessing
- supervised learning
- unsupervised learning
- model comparison
- neural networks
- reinforcement learning
- deep learning
- evaluation and reflection

Among the supervised learning approaches, SVM achieved the strongest classical result, while the deep learning notebook also produced strong performance. The reinforcement learning notebook provided a clear example of learning through interaction and showed the importance of parameter tuning.

Overall, the portfolio shows not only working final results, but also the process of development, experimentation, debugging, and improvement across multiple notebook-based projects.

12. References

Narwade, A. (n.d.) AI Impact on Student Performance. Kaggle. Available at: <https://www.kaggle.com/datasets/ankushnarwade/ai-impact-on-student-performance> (Accessed: 24 April 2026).

scikit-learn Developers (n.d.) KNeighborsClassifier. scikit-learn documentation. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html> (Accessed: 24 April 2026).

scikit-learn Developers (n.d.) KMeans. scikit-learn documentation. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html> (Accessed: 24 April 2026).

scikit-learn Developers (n.d.) SVC. scikit-learn documentation. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html> (Accessed: 24 April 2026).

scikit-learn Developers (n.d.) MLPClassifier. scikit-learn documentation. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html (Accessed: 24 April 2026).

Farama Foundation (n.d.) Frozen Lake. Gymnasium documentation. Available at: https://gymnasium.farama.org/environments/toy_text/frozen_lake/ (Accessed: 24 April 2026).

Farama Foundation (n.d.) Solving FrozenLake with Tabular Q-Learning. Gymnasium tutorials. Available at: https://gymnasium.farama.org/v1.1.0/tutorials/training_agents/FrozenLake_tuto/ (Accessed: 24 April 2026).

TensorFlow (n.d.) The Sequential model. TensorFlow documentation. Available at: https://www.tensorflow.org/guide/keras/sequential_model (Accessed: 24 April 2026).

TensorFlow (n.d.) tf.keras.layers.Dense. TensorFlow documentation. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense (Accessed: 24 April 2026).

TensorFlow (n.d.) tf.keras.layers.Dropout. TensorFlow documentation. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout (Accessed: 24 April 2026).